Kafka Keys, Partitions and Message Ordering

Rob Golder - July 2023

Introduction

In order to guarantee ordered message delivery with the Kafka messaging broker, messages can be produced with a key. Messages with the same key are written to the same topic partition, and as consumers read messages in order from each partition, their ordering is preserved.

A Spring Boot application that demonstrates writing messages with keys and the impact on ordering accompanies this article. The full source code is available here.

Topic Partitions

Each Kafka topic is divided into one or more partitions. This means that when a producer is writing a message to a topic, it will write it to one of the partitions in the topic. Likewise a single consumer in a consumer group that is subscribed to the topic will consume from all the partitions.

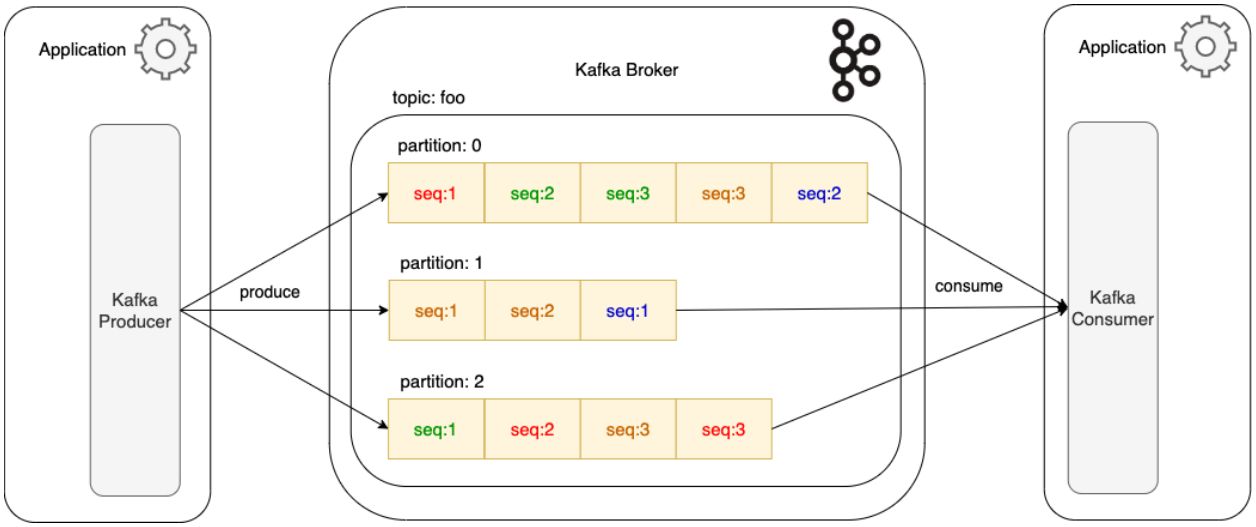

Figure 1: Single producer and consumer

In this example, messages are being produced without a message key, so there is no guarantee as to which topic partition they will be written to. Related messages have been colour coded, and we see that messages relating to the red entity have been written to partition 0 and 2. As a result, the consumer polling for batches of records from the partitions will receive these related messages in an unknown order. In this case, it may receive later red messages written to partition 2 before it receives the first red message written to partition 0.

Scalability

Topic partitions are a core aspect of the high scalability that the Kafka messaging broker provides. By increasing the number of partitions in a topic in line with the number of consumer instances in the same consumer group that are subscribed to this topic, throughput is increased. Whereas in figure 1 there is a single consumer reading from all three topic partitions, in figure 2 there are now multiple consumer instances reading from the topic. The topic partitions are divided up between the consumer instances, so the throughput has increased.

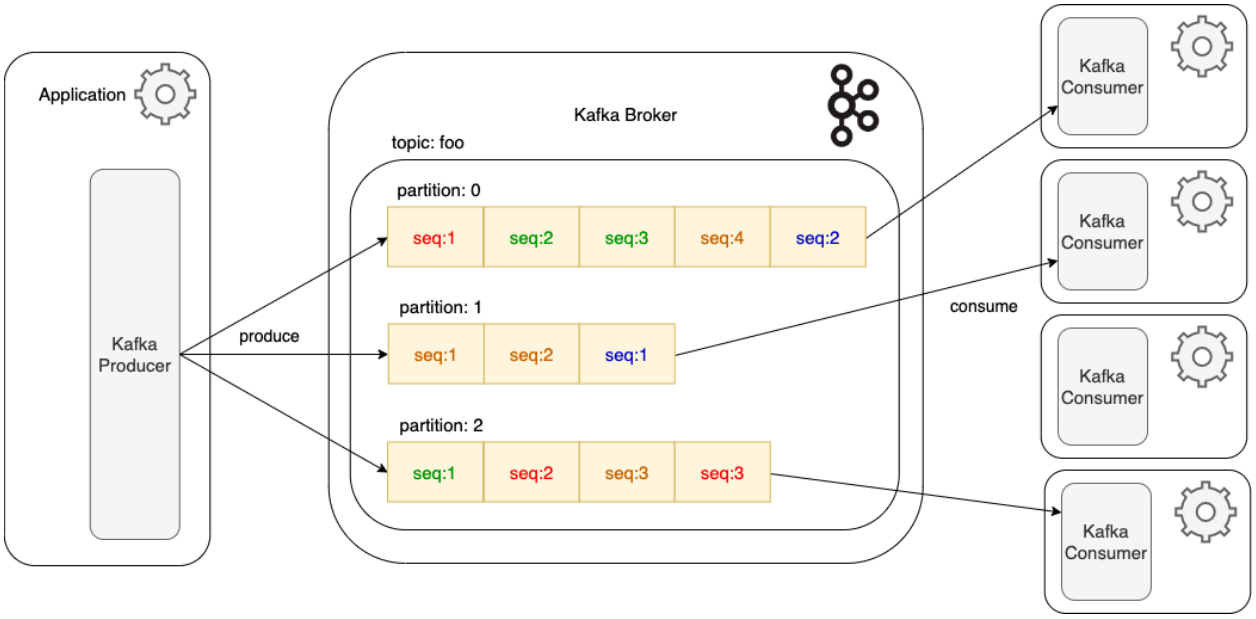

Figure 2: Multiple consumers

This diagram highlights the fact that related messages written across different partitions will be consumed and processed in an unknown order. Messages for the red entity will be consumed by two different consumer instances, those assigned to partition 0 and partition 2, so it is unknown which messages will be processed first.

In figure 2 there are four consumers, but with only three topic partitions it means that the fourth consumer instance is idle as it has no free partition to be assigned to. This could be deemed a waste of resources as the consumer instance is still using CPU and memory. On the other hand it means that if one of the other consumer instances fails, the spare consumer would be available on the resulting consumer group rebalance, minimising the impact of the failed consumer.

While one consideration might be to increase the topic partition count to four to mean that all available consumer instances would be assigned partitions, this is not a recommended practice. Topic partition counts should be carefully sized in advance to cater for current and expected data volumes. Increasing the number of partitions once a topic is in Production can result in a loss of order, and the number of partitions can not be reduced, as data would be lost.

Message Keys

In order to guarantee message ordering, related messages that should be ordered must be written to the same topic partition. This is achieved by writing the message with a header key. There is a less common alternative to using a message key which is for the producer itself to stipulate which partition to write a message to.

As with the message body, the key can be typed as required, such as a String, Integer or a JSON object. Likewise, as with the body, the key can have an Avro schema defined. The key is hashed by the producer, and this hash is used to determine which partition to write the message to. If no message with this hashed key has yet been written to a partition, (or indeed no key has been included) then a new partition will be selected, with Kafka typically attempting to spread load evenly across the partitions.

Message Ordering

Returning to the above example, in the following diagram four different keys each representing four different entities have been written to a topic with three partitions.

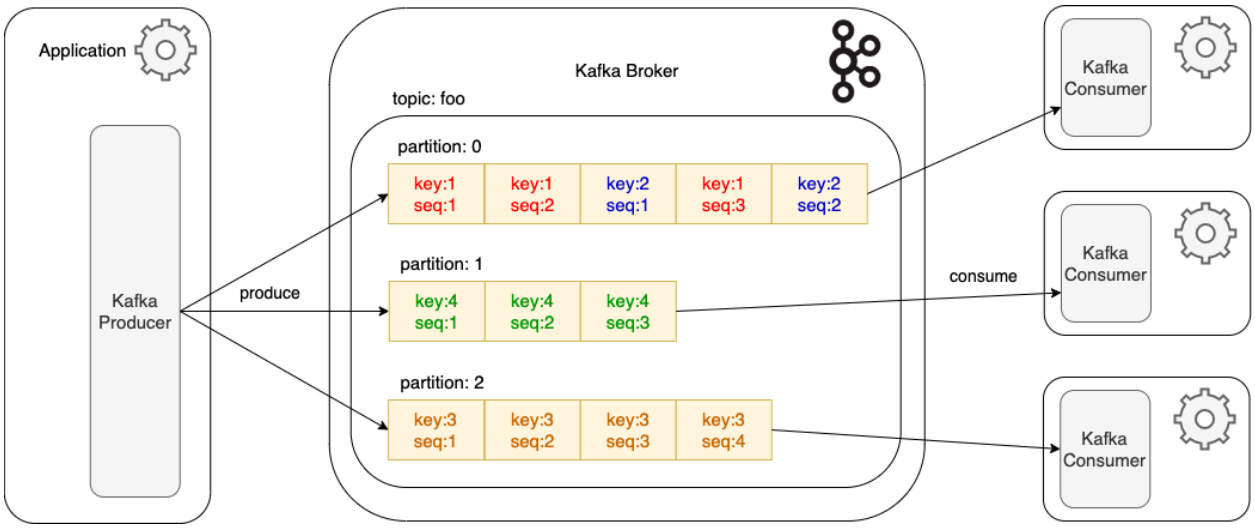

Figure 3: Producing keyed messages

This time all messages with the same key are written to the same partition. Of course any one partition will still hold messages for different keyed messages, as we see with messages for the red and blue entities both being written to partition 0. Now however when a consumer receives messages from a partition those messages relating to each entity will be in a guaranteed order.

Concurrent Consumers

Spring Kafka makes it straightforward to instantiate multiple instances of an application's Kafka consumer from within the single application deployment. Each of these consumer instances are treated as separate consumers in the consumer group, so the topic partitions will be assigned between them, as with any other consumers in the same consumer group. They operate concurrently, so if memory and CPU allow this is another good option for increasing throughput.

In the accompanying Spring Boot application an example consumer KafkaDemoConsumer consumes messages from the topic demo-inbound-topic. The consumer is created using Spring's KafkaListenerContainerFactory, and this Spring bean is defined in KafkaDemoConfiguration. The ConcurrentKafkaListenerContainerFactory implementation can be configured to create multiple concurrent instances of each consumer it creates. In the example the concurrency has been configured to 3:

```
ConcurrentKafkaListenerContainerFactory factory = new
ConcurrentKafkaListenerContainerFactory<>();
factory.setConcurrency(3);
```

The upshot is that when the application is started then three instances of the KafkaDemoConsumer are started. The assignment of the instances across the topic partitions can be viewed using the Kafka command line tools, specifically using the kafka-consumer-groups tool to describe the consumer group. The consumer group is named demo-consumer-group as configured on the @KafkaListener annotation on the KafkaDemoConsumer.

```
kafka-consumer-groups --bootstrap-server localhost:9092 --describe --group
demo-consumer-group
```

```
[appuser@kafka ~]$ kafka-consumer-groups --bootstrap-server localhost:9092 --describe --group demo-consumer-group

GROUP               TOPIC               PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG             CONSUMER-ID                                                   HOST            CLIENT-ID
demo-consumer-group demo-inbound-topic 0          0               0               0               consumer-demo-consumer-group-1-d0a6d60c-e61e-45ab-b5ee-c80c8955091e /172.19.0.1     consumer-demo-consumer-group-1
demo-consumer-group demo-inbound-topic 1          0               0               0               consumer-demo-consumer-group-1-d0a6d60c-e61e-45ab-b5ee-c80c8955091e /172.19.0.1     consumer-demo-consumer-group-1
demo-consumer-group demo-inbound-topic 2          0               0               0               consumer-demo-consumer-group-1-d0a6d60c-e61e-45ab-b5ee-c80c8955091e /172.19.0.1     consumer-demo-consumer-group-1
demo-consumer-group demo-inbound-topic 3          0               0               0               consumer-demo-consumer-group-1-d0a6d60c-e61e-45ab-b5ee-c80c8955091e /172.19.0.1     consumer-demo-consumer-group-1
demo-consumer-group demo-inbound-topic 4          0               0               0               consumer-demo-consumer-group-2-7ba81e5c-adee-47a9-a071-555114b1b370 /172.19.0.1     consumer-demo-consumer-group-2
demo-consumer-group demo-inbound-topic 5          0               0               0               consumer-demo-consumer-group-2-7ba81e5c-adee-47a9-a071-555114b1b370 /172.19.0.1     consumer-demo-consumer-group-2
demo-consumer-group demo-inbound-topic 6          0               0               0               consumer-demo-consumer-group-2-7ba81e5c-adee-47a9-a071-555114b1b370 /172.19.0.1     consumer-demo-consumer-group-2
demo-consumer-group demo-inbound-topic 7          0               0               0               consumer-demo-consumer-group-3-91b35ced-805c-43d3-a45f-6a4115eb7ec1 /172.19.0.1     consumer-demo-consumer-group-3
demo-consumer-group demo-inbound-topic 8          0               0               0               consumer-demo-consumer-group-3-91b35ced-805c-43d3-a45f-6a4115eb7ec1 /172.19.0.1     consumer-demo-consumer-group-3
demo-consumer-group demo-inbound-topic 9          0               0               0               consumer-demo-consumer-group-3-91b35ced-805c-43d3-a45f-6a4115eb7ec1 /172.19.0.1     consumer-demo-consumer-group-3
```

Figure 4: Partition assignment

In this example there are 10 topic partitions for the demo-inbound-topic, and a single instance of the demo Spring Boot application has been started. The partitions can be seen to have been assigned across the three concurrent consumer instances within the one consumer group. So long as related messages are written to the topic with the same key they will be produced to the same partition, and hence consumed in order by the assigned consumer instance.

Producer Retries

It is common practice to configure a producer to retry writes that fail due to transient errors, such as network connection failures. By default retries are set to the maximum integer value. However retrying can impact message ordering guarantees, depending on other producer configuration values.

The configuration parameters of interest are the following:

Producer Config Param	Usage	Default
retries	Cause the producer to retry transient write failures	2147483647
enable.idempotence	When true, ensures that the message is only ever written once	true
max.in.flight.requests.per.connection	The maximum unacknowledged requests before the producer blocks	5

Message ordering is no longer guaranteed if retries occur due to transient errors with the following combinations of configuration settings:

```
retries > 0
enable.idempotence = false
```

```
max.in.flight.requests.per.connection > 1
```

With enable.idempotence set to false, if the first batch request fails with a transient error and is retried, but a second in-flight batch is written successfully to the same topic partition before the first is successfully completed, the messages will have been written out of order.

```
retries > 0
enable.idempotence = true
```

```
max.in.flight.requests.per.connection > 5
```

With this second combination of configuration parameters where enable.idempotence is set to true, an attempt to perform a duplicate write to Kafka would on the face of it be stopped. However this guarantee is lost if max.in.flight.requests.per.connection is greater than 5. The limit of 5 here is the highest supported value that means a producer can be idempotent. There is more detail on the idempotent producer in the article [Kafka Idempotent Producer](#).

Adding Or Removing Partitions

If the number of partitions for a topic is changed, then ordering guarantees are lost for new messages with a given key compared to existing messages with the same key written prior to the change. The partition selected for a given key will be based upon the current number of partitions. So as that number has increased or decreased, there is no guarantee that new messages with a particular key will be written to the same partition as existing messages with that same key - and hence guaranteed ordering is lost. Of course all new keyed messages written following a change to the number of partitions will be guaranteed to be in order with other new keyed messages, just not existing ones. Meanwhile if the number of partitions is decreased then any messages on the partitions that are removed will be lost.

Spring Boot Demo

Overview

The accompanying Spring Boot application is used to demonstrate the impact of adding a key to a message and the partition it is written to. The application consumes keyed messages from a topic named demo-inbound-topic. This results in an outbound keyed message being written to a topic named demo-outbound-topic.

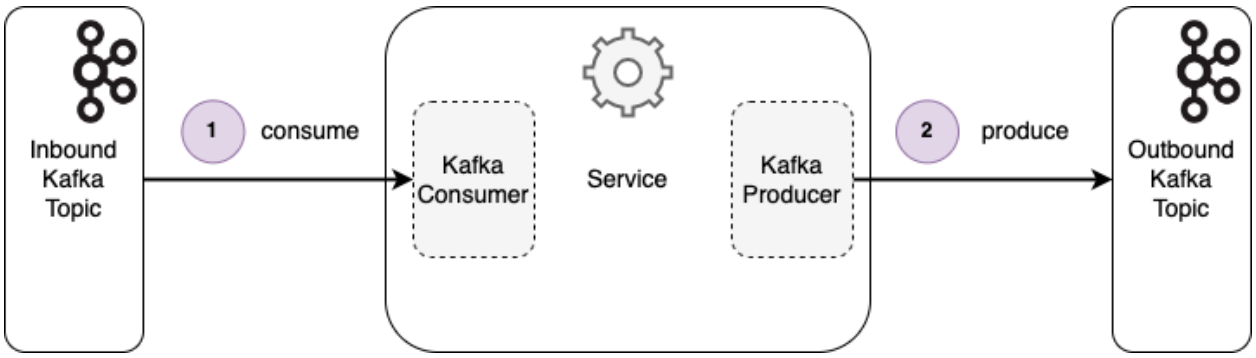

Figure 5: Spring Boot demo application

Consumer

As demonstrated in the KafkaDemoConsumer, using Spring Kafka the key of the consumed message is passed to the @KafkaListener message handler when a method argument is provided with the KafkaHeader named RECEIVED_KEY.

```
@KafkaListener(
        topics = "demo-inbound-topic",
```

```
        groupId = "demo-consumer-group",
        containerFactory = "kafkaListenerContainerFactory")
public void listen(@Header(KafkaHeaders.RECEIVED_PARTITION) Integer
partition, @Header(KafkaHeaders.RECEIVED_KEY) DemoInboundKey key, @Payload
DemoInboundPayload payload) {
```

Likewise, the topic partition that the message was consumed from can also be obtained, via the KafkaHeader named RECEIVED_PARTITION.

Producer

The KafkaDemoProducer demonstrates sending a message with a message key using Spring Kafka's KafkaTemplate. The KafkaTemplate method send() is overloaded, and by simply including the key in this call Spring Kafka will include the key in the message it writes to Kafka.

```
kafkaTemplate.send(properties.getOutboundTopic(), key, event);
```

Testing

In order to prove the behaviour, an integration test KafkaIntegrationTest using Spring Kafka's embedded broker is provided. This is configured with 10 partitions per topic, and 10,000 messages are sent to the inbound topic with 5 unique keys. The test includes a test listener that tracks the partitions that each resulting outbound keyed message is received on, along with the message sequence. It then verifies that each message key is only ever received on the same partition, and that each message for a given key has an ascending sequence number, proving that message ordering is preserved.

Similarly a component test EndToEndCT that spins up Docker containers for Kafka, Zookeeper, and the application under test is provided. Using a real Kafka instance this time, once again the test verifies that each message key is only ever received on the same partition.

Kafka Command Line Tools

Kafka provides a set up command line tools for interacting with the broker, and these include tools that enable sending and receiving messages to and from Kafka topics. These are demonstrated in the accompanying Spring Boot application. Of particular note here is how to include a message key when sending a message, and how to view a message key when consuming a message on the command line.

The demo application can be run up, first building with maven, then bringing up Kafka and Zookeeper in Docker using the provided docker-compose.yml, and finally executing the application jar:

```
mvn clean install
docker-compose up -d
```

```
java -jar target/kafka-message-keys-1.0.0.jar
```

The Kafka Docker container contains the command line tools, so jump on to the container with:

```
docker exec -ti kafka bash
```

The usual command to produce a message with the kafka-console-producer is provided two extra properties, parse.key, set to true, and key.separator, set to the character to use as the separator in the provided message, for example a semi-colon.

```
kafka-console-producer \
--topic demo-inbound-topic \
--broker-list kafka:29092 \
--property parse.key=true \
```

```
--property "key.separator=;"
```

The message is now entered on the command line, and this includes the key, which in this example is a JSON formatted String with an id field set to "123", followed by the semi-colon, and then the JSON payload:

```
{"id":"123"};{"inboundData": "my-data", "sequenceNumber": 1}
```

The demo application consumes this message from the demo-inbound-topic, and emits a keyed message to the demo-outbound-topic which can be consumed using the kafka-console-consumer. This command also has two extra properties included to view the key, print.key, set to true, and key.separator. In order to view messages already written to the topic, the --from-beginning parameter is also included:

```
kafka-console-consumer \
--topic demo-outbound-topic \
--bootstrap-server kafka:29092 \
--from-beginning \
--property print.key=true \
```

```
--property key.separator=";"
```

The following outbound message is displayed:

```
{"id":123};{"outboundData":"Processed: my-data","sequenceNumber":1}
```

Conclusion

Kafka guarantees message ordering when related messages are written to Kafka with the same key. Messages with the same key are written to the same topic partition. As each partition is assigned to a single consumer instance within a consumer group, and messages are consumed in order from each partition, then ordering is preserved for messages with the same key. However, if producer retries are enabled for transient errors then it is important to verify the associated configuration parameters to ensure the ordering guarantee remains.

Source Code

The source code for the accompanying Spring Boot demo application is available here:

https://github.com/lydtechconsulting/kafka-message-keys/tree/v1.0.0